

MoveProof

Assessment Task 3: Project 2: Platform-Integrated Solution

Subject Code: 40005

Subject Name: Advanced iOS Development

Student: Kai-Hsiang Chuang

Student ID: 25605484

Table of Contents

1. Problem Statement.....	3
1.1 What is the problem?	3
1.2 Primary stakeholder	3
1.3 Consequences for the stakeholder	3
1.4 Real-world evidence	4
1.5 Scope of the solution	5
2. Design Justification	6
2.1 Why an iOS Application?	6
2.2 Why These System Extensions?	6
2.2.1 WidgetKit Widget.....	6
2.2.2 Share Extension	7
2.3 Why Core Data?	8
3. Architecture Diagram	10
4. Reflective Report	12
4.1 Problem and Justification	12
4.2 Extension Design Decisions	12
4.3 Database Design Decisions.....	12
4.4 Architecture Under Pressure.....	13
4.5 Use of AI Tools	13
References.....	14

1. Problem Statement

1.1 What is the problem?

A renter moving into a property in New South Wales has seven days from the day they move in to complete the condition report supplied by the landlord or agent and return one copy to them (NSW Government, n.d.). That week is also the week the renter is unpacking, arranging utilities and usually still working. The record they assemble in those seven days is the only account they will hold of what the property looked like before they lived in it.

The difficulty is not that renters fail to take photographs. It is that the photographs never become a record. A renter typically finishes moving week with evidence spread across four separate places: the camera roll, mixed in with everything else from the move; the condition report itself, as a PDF sitting in an email thread; messages to the agent about one particular fitting; and screenshots of the listing or of earlier correspondence. Nothing in that collection records which room a photograph was taken in, and nothing connects a photograph to the line on the condition report it was meant to support.

The consequence is that a set of ordinary questions becomes unanswerable, both during the walkthrough and months later. Which rooms have actually been checked. Which room a given photograph belongs to. Whether there is anything at all backing up a box that was ticked as damaged. Whether the inspection is finished, or whether four rooms were skipped because the renter ran out of evening.

One of those failures is worse than the others, and it is the one MoveProof is designed around: damage recorded with nothing to support it. A tick in a damaged box, carrying neither a note nor a photograph, is indistinguishable at the end of a tenancy from a tick made by accident. The renter cannot say what they saw, where it was, or how bad it looked. They did the work and still hold nothing they can use. The same failure repeats whenever something is documented later in the tenancy, because the evidence captured then lands in the same unstructured pile as the evidence captured on day one.

1.2 Primary stakeholder

The primary stakeholder is a residential renter documenting the condition of a rental property during a tenancy. More precisely, this is a renter in New South Wales who is working to the seven day condition report deadline at the start of a tenancy, and who continues to record the property's condition for the rest of it.

Two characteristics of this person shaped the design directly. The first is when they are using the application. They are using it on moving day, tired, surrounded by boxes and working to a deadline, so MoveProof opens with inspection areas and condition items already seeded rather than asking the renter to build a checklist before they can record anything. The second is what the data is. The evidence describes where this person lives and what is wrong with it, which is why the widget was designed around what it must not show as much as around what it does show, as Section 2.2.1 explains.

1.3 Consequences for the stakeholder

The cost of leaving this problem unsolved falls on the renter in small, repeated ways rather than in a single dramatic event.

The most immediate cost is time. Evidence that is not filed has to be found, and finding a particular photograph in a camera roll that also holds the move itself means scrolling by date and guessing. The second cost is that photographs become disconnected from the room and the condition item they document. A photograph of a scratched floorboard that cannot be tied back to a specific room, in a specific property, at the start of a specific tenancy, carries much less weight than the same photograph filed against the line of the condition report it supports.

The third cost is a weak evidence trail. Where the capture time is not recorded alongside the evidence, the renter cannot reconstruct when something was documented, which matters because the point of a condition report is to establish the state of the property at a particular moment. The fourth is that incomplete documentation is hard to review. Without a view of what has been covered, a renter cannot tell which rooms were missed, and the gap is usually discovered long after the deadline has passed. The fifth is retrieval friction later in the tenancy, when evidence is needed to discuss a repair or to complete the outgoing report and the renter has to reassemble a year old collection from memory.

This problem is also connected to my own rental experience. In a previous tenancy I had a disagreement with the landlord about the condition in which the property was returned. The landlord considered it insufficiently cleaned and deducted money from our bond. That experience made the value of a clear and organised record of a property's condition much more concrete to me, particularly where the two parties remember the same situation differently.

NSW Government (n.d.) states that the condition report can be used as evidence if there is a disagreement about missing items or damage, and the Tenants' Union of NSW (2025) states that the report will be used as evidence if the landlord or agent disputes the return of the bond. MoveProof improves the organisation and retrievability of that evidence. It does not determine liability and makes no claim about the outcome of any dispute.

1.4 Real-world evidence

Two sources establish that the problem is real and bounded rather than assumed.

NSW Government (n.d.) sets out the obligation the stakeholder is working to. The page states that the renter must “return one completed copy to the landlord or agent within seven days of moving in”. It also advises renters to “take photos to go with the condition report, making sure to include time stamps to show when images were taken”, and states that “the more detail, including time-stamped photographs, in a condition report, the less likely disputes will arise at the end of a tenancy”. Finally it states that “if there is a disagreement or dispute about missing items or damage, the condition report can be used as evidence”.

Two things follow for the design. The first is that supporting evidence is officially recommended rather than optional in practice, so an application that organises it is addressing a documented expectation. The second is that the deadline is specific, which is why seven days is a domain constant in MoveProof rather than a number in the interface. The application proposes a due date seven days after the move in date and allows the renter to change it, and presents it as a default rather than as advice.

Tenants' Union of NSW (2025) describes the same process from the renter's side. It states that the landlord or agent “must give you 2 copies, one for you to keep and one for you to return to them”, that “the report will be used as evidence if the landlord/agent disputes the return of your bond at the end of the tenancy”, and that “it is also a good idea to take photographs at the start (and the end) of the

tenancy, and store them in a safe place”. The last of those is the clearest statement of the gap MoveProof addresses: the advice is to store photographs safely, but the advice stops short of saying how they stay connected to the report they support.

1.5 Scope of the solution

MoveProof is an evidence capture, organisation and retrieval tool for one renter and one property at a time. It does not give legal advice, does not determine who is responsible for any damage, and makes no claim about the outcome of a dispute. The seven day due date is taken from the NSW Government guidance cited above and is offered to the renter as a changeable default rather than as legal advice.

2. Design Justification

2.1 Why an iOS Application?

The decision to build a native iOS application rather than a website, a desktop application, a folder in a cloud drive or a photo album follows from where and when the evidence is created.

The evidence is created on the device, in the property, while the renter is standing in front of the thing they are recording. The photographs are already on the phone, so the application that files them should be on the same phone. A web application would insert an upload step between noticing the damage and recording it, at exactly the moment the renter has the least patience for extra steps. A desktop application would defer the whole task until the evening, by which point the renter is recalling rooms rather than looking at them. MoveProof attaches evidence to a condition item through SwiftUI's PhotosPicker, which reads from the device photo library where the renter's photographs already are. The application does not implement its own camera capture, so the claim here is specifically about proximity to the photographs, not about owning the capture pipeline.

Generic cloud storage and the system photo gallery both solve storage and neither solves structure. A shared folder will hold a hundred photographs safely and still cannot answer which condition item a given photograph supports, or which rooms have no evidence at all. The missing element is the relationship between a photograph and the line of the condition report it documents, and that relationship is what MoveProof stores.

Three platform capabilities that MoveProof actually uses only exist on the device. The system Share Sheet lets evidence arrive from Mail, Files and Photos without the renter re-finding it inside MoveProof, which a website cannot receive. WidgetKit puts the deadline and the progress count on the Home Screen, where the renter sees it during the week it matters without opening anything. The App Group shared container lets the main application, the Share Extension and the Widget Extension cooperate through one piece of storage while staying in separate processes.

Finally, the data is private, local and single user. One renter, one property, one device. MoveProof contains no networking code at all, so the evidence never leaves the device and the application works in a basement flat with no signal, which is a realistic place to be doing an inspection. A web application would require the opposite by construction.

2.2 Why These System Extensions?

MoveProof implements two system extensions: a WidgetKit widget extension and a share extension. Each is justified below by the renter's situation rather than by the capability itself.

2.2.1 WidgetKit Widget

During the seven day window the renter has a clock running, and the question they return to is not what they recorded in the kitchen. It is whether they are on track and how long they have left.

Without the widget, answering that question costs an app launch, a wait for the dashboard to load, and a decision to go and look in the first place. That last cost is the expensive one. A renter who has to decide to check their progress checks it less often, and in a seven day window the discovery that two rooms were never started is worth having on day three rather than day six.

The widget removes that cost by making the answer ambient. It exposes exactly the domain information that answers the recurring question and nothing else: how many inspection areas are complete out of the total, how many areas contain damage recorded without supporting detail, how much evidence has been filed, how many shared items are still waiting in the inbox, and how many days remain until the condition report is due. The small family answers the two number question of areas reviewed and days left; the medium family adds what needs attention and what is waiting to be filed. Both families are declared through supportedFamilies with `.systemSmall` and `.systemMedium` (Apple, n.d.-b).

The widget reads its data from the App Group shared container. The main application writes a reduced summary to `Widget/snapshot.json` inside `group.com.krischuang.MoveProof`, and the widget extension reads that file. It never touches Core Data, which keeps a tightly budgeted extension process away from a persistence stack it has no reason to load. That shared container is also what makes the privacy boundary enforceable, because the snapshot type has no field capable of carrying the property address, a note or a file name, so the sensitive part of the domain cannot reach shared storage even by accident.

The widget has been verified in two ways. The automated tests render both families and assert they are not blank, including a stress case with three digit counts and an overdue deadline; they assert that a snapshot written by the application is readable through the widget's own reading path; and `WidgetRefreshAfterChangeTests` covers the republish that follows a successful write.

In addition to the automated tests, I manually verified the widget on the iOS Home Screen. Both the small and medium families could be added successfully and displayed the expected MoveProof data. I also confirmed that relevant changes made in the main application were reflected by the widget once the shared snapshot had been refreshed. No problems were found during that manual check.

2.2.2 Share Extension

The renter frequently already holds the relevant file somewhere else. The condition report the agent sent is a PDF in Mail. A photograph of a water stain is in Photos. A receipt for a repair is in Files. In each case the renter is looking at the file, in another application, and wants it to become evidence in MoveProof.

Without a share extension the only route in would be MoveProof's own importer, which means opening MoveProof, navigating to the right screen, opening a file browser and finding the file again. That sequence recreates the exact problem the application exists to solve. The renter already knows where the file is at the moment they are looking at it, and the importer throws that knowledge away and asks them to rediscover it from inside a different application.

The share extension lets the renter push the file in from wherever it already lives, at the moment they are already looking at it. MoveProof appears in the system Share Sheet for images and files, presents a short confirmation, and the file is handed over.

What the extension writes is deliberately limited. It copies the file into the `SharedEvidenceInbox` directory of the App Group container and writes a small JSON record beside it holding the original file name, the content type identifier reported by the host application, the time it was received and the size on disk. It records no tenancy, no inspection area, no condition item and no evidence type, because it is not the part of the system that decides those things.

The main application consumes the inbox on its own schedule. The Shared items screen lists what is waiting, and `ImportSharedEvidenceUseCase` decides whether each item is usable, which tenancy it

belongs to, and whether it has already been imported, then adopts the file into the application's own evidence directory and records the metadata in Core Data. Only after that has succeeded is the inbox entry removed, so an interruption part way through leaves the item waiting rather than losing it. For the renter the effect is that evidence arrives where the rest of their evidence is, without a detour through a file browser.

2.3 Why Core Data?

MoveProof stores its domain data in Core Data, Apple's object graph and persistence framework (Apple, n.d.-a), rather than in CloudKit. The choice follows from what this application is for rather than from a general ranking of the two frameworks.

The data is private, local and single user. One renter is documenting one property on one device. There is no collaboration requirement, no second editor and no server that needs to hold evidence describing where someone lives. CloudKit is designed for the opposite situation, where the same records are reached from several devices and conflicting edits have to be reconciled. Adopting it here would introduce an iCloud account requirement and a conflict resolution problem in exchange for a capability the stakeholder has not asked for, and it would place the renter's evidence on a server when the current design never sends it anywhere. Core Data is the better fit for MoveProof's current scope, which is a local, private, single user record. If the scope later grew to include sharing a tenancy with a co-tenant, that trade would be worth revisiting.

Two further requirements point the same way. The renter is frequently offline, in a property with poor reception, and the inspection cannot pause for a network round trip. And the queries have to be fast enough to run while a screen is loading, because the dashboard recalculates progress every time it appears.

The structure is the main reason, though. MoveProof stores four entities that mirror how a condition report is actually organised: a Tenancy holds InspectionAreas, each InspectionArea holds ConditionItems, and a ConditionItem holds EvidenceItems. Evidence is also related directly to the tenancy, so a photograph that is not yet filed against a condition item still belongs to a property.

The relational structure matters to the renter rather than only to the schema. Core Data keeps each piece of evidence explicitly connected to the tenancy, inspection area and condition item it documents, which is the connection the camera roll loses. A photograph without its inspection context is a picture of a floorboard. The same photograph filed against the condition item it supports, in the room it was taken in, at the start of a named tenancy, is evidence.

That structure is also what makes the application's central question answerable. The query MoveProof is built around asks which condition items in this tenancy record damage or a fault but carry neither a note nor a single piece of evidence. As a predicate it traverses two relationships to reach the tenancy, filters on the recorded condition state, requires empty notes, and aggregates the evidence relationship with an @count comparison. The store answers it in one fetch. The alternative, loading every condition item in the tenancy and filtering in Swift, would produce the same answer while doing the work a relational store exists to do. Five further predicate queries follow the same pattern, including the active tenancy lookup, the unfiled evidence filter that combines a relationship traversal with a null check, and the duplicate check that makes importing shared evidence idempotent.

One thing is deliberately kept out of the store. The evidence files themselves are not stored in Core Data. Photographs are large and a renter may file dozens, so they live in the Evidence directory of the App Group container while Core Data holds the metadata and a relative file name. This keeps the

store small, lets the file system do what it is good at, and means a file handed over by the share extension can be adopted in place rather than copied out of the container and back into it.

3. Architecture Diagram

Figure 1 shows the complete architecture: the boundary between the renter and the system, the four layers of the main application, both extension processes, the App Group shared container and where Core Data persistence occurs. The numbered arrows follow one real workflow from end to end, which is the renter attaching a photograph of a scratched floorboard to a condition item and the Home Screen widget catching up with it.

Reading down the centre, each arrow inside the main application is a boundary the code may not skip. A view holds no storage and no business rules, a view model holds screen state only, every business rule lives in one of the thirteen use case structs, and the use cases reach storage only through repository protocols written in domain value types. The one shortcut the design allows is drawn as the dashed arrow on the right: loading a screen reads through those same repository protocols without a use case in between, because a read enforces no rule and changes nothing. Every write and every business rule still goes through a use case, and no view and no view model imports CoreData or holds an `NSManagedObjectContext` in either direction. Refusals travel back the other way as typed domain errors that carry the text the renter reads.

The right hand side of the diagram shows the platform integration. The main application is the only writer of the Evidence and Widget directories, the share extension is the only writer of `SharedEvidenceInbox`, and the widget extension only reads. Step 7 is the coordination point discussed in Section 4.4: the widget snapshot is republished through `MoveProofModel.dataChanged()` after a write has committed, and only after it.

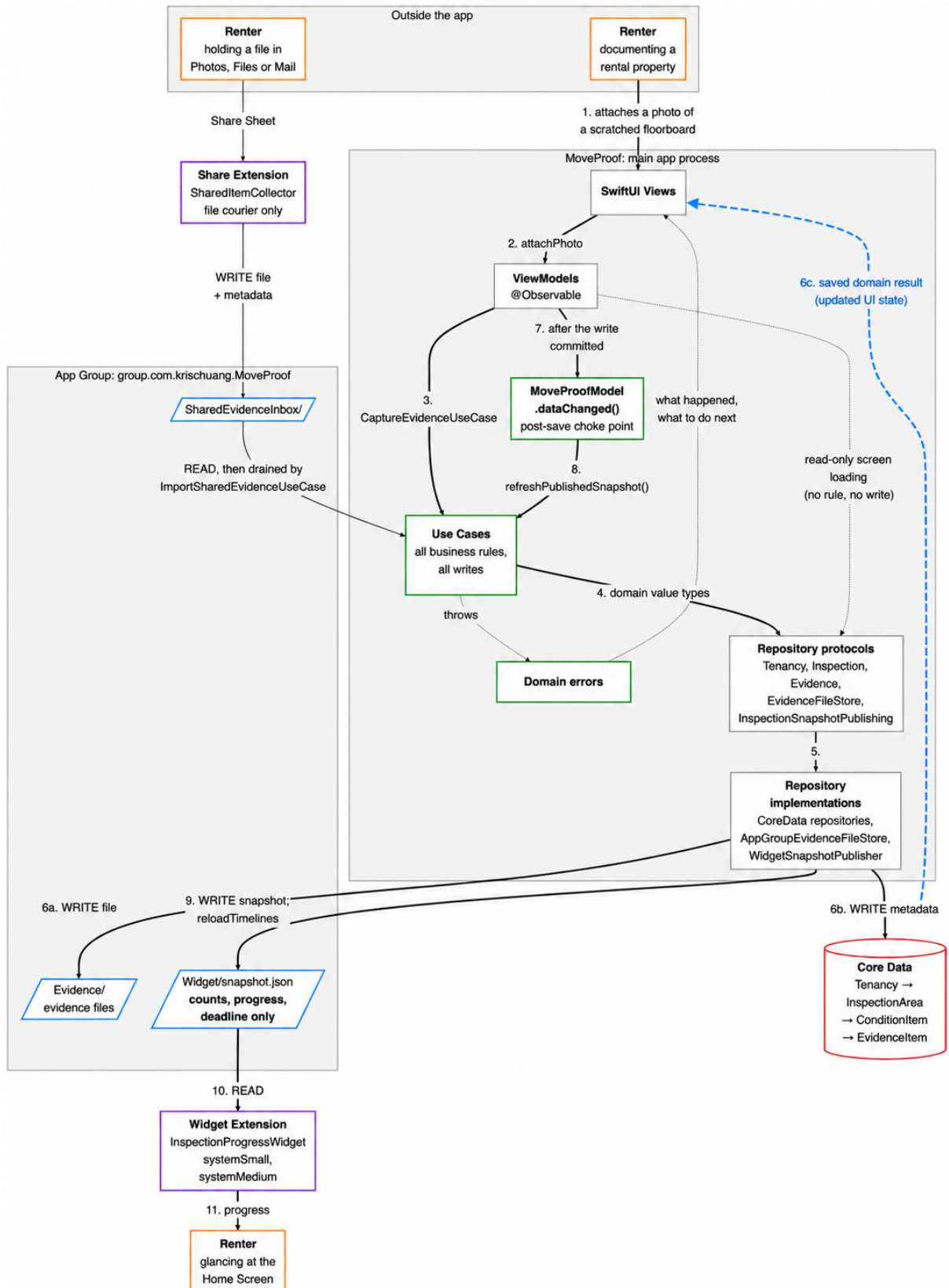


Figure 1. MoveProof platform-integrated architecture and primary evidence-capture data flow.

4. Reflective Report

4.1 Problem and Justification

I chose rental condition evidence because the failure is specific and anchored to a real obligation: the seven day condition report deadline, with time-stamped photographs recommended to support it (NSW Government, n.d.).

The problem also had a personal basis. In an earlier tenancy I had a disagreement with a landlord who believed the property had not been cleaned adequately and deducted money from our bond. What stayed with me was less the outcome than the difficulty of reconstruction: once a tenancy has ended, the condition a property was actually in is hard to establish from memory alone. That pushed me towards designing MoveProof around preserving evidence with enough context for it to still be understood later, rather than around another place to put photographs.

I began by treating MoveProof as organised storage, but writing the use cases made clear that storage was never the failure: renters already take photographs, and the failure is that they never attach to anything that gives them meaning. That narrowed the design to one case the application could refuse rather than encourage, which is damage recorded with nothing to support it.

4.2 Extension Design Decisions

The test I applied to both extensions was whether the renter would notice their removal.

The widget would be noticed during the week it matters, because without it progress and the deadline are only reachable by opening the application. The tension was glanceable usefulness against privacy, since the most useful content would be the address, the outstanding damage and a photograph. I resolved it in the model rather than at display time, so `InspectionSnapshot` cannot hold an address, a note or a file name. The cost is that the medium widget says two areas need attention without saying which, which I accepted because its job is to prompt, not inform.

The share extension would be noticed more sharply, because fragmentation is the problem itself. Writing straight into the domain would have been less code, but it would duplicate the import rules into a short lived, memory constrained process, which is the worst place for logic that matters. Leaving every judgement to `ImportSharedEvidenceUseCase` means one place decides what counts as evidence.

4.3 Database Design Decisions

Core Data was chosen because MoveProof's questions are relational and local; its central query spans two relationships and aggregates a third. The most significant schema decision, though, was the delete rules rather than the entities. A tenancy cascades to its areas and evidence, and an area to its condition items, because those children are meaningless alone. The condition item to evidence relationship nullifies instead, so removing an area destroys the checklist but leaves the photographs as unfiled evidence.

The trade-off is protecting evidence against keeping filing simple. Cascading throughout would be easier to reason about, but it would let one mis-tap destroy a photograph that cannot be taken again. Nullifying makes unfiled evidence a real state the application must handle, which costs a library filter and a count in `RemoveInspectionAreaUseCase` so the renter knows how much is now unfiled.

4.4 Architecture Under Pressure

The sharpest conflict between platform requirement and clean architecture was widget synchronisation. `WidgetKit` must be told when to reload, and the layer that knows a write succeeded is the domain layer, which should not know `WidgetKit` exists.

The snapshot was originally published as a side effect of `ReviewInspectionProgressUseCase`, which the dashboard runs when it loads. That was correct whenever the renter returned to the dashboard and wrong the rest of the time: signing off a room left the Home Screen stale.

Calling `WidgetCenter` from each use case would be reliable, but it would put a platform framework inside every business rule and make the refresh something each new use case has to remember. Leaving it screen driven keeps the domain clean and accepts stale data. I took a third option: `MoveProofModel.dataChanged()` is the one place the application records a save, so the snapshot is republished there after the commit.

That buys one place to reason about platform consistency instead of thirteen, with the domain layer unchanged. The cost is that a convention is not compiler checked, so a future call site that saves and forgets would reintroduce the staleness; tests cover it.

4.5 Use of AI Tools

Claude Code was the main AI-assisted development tool used on this project. It scaffolded the Xcode targets for both extensions, drafted domain models, use cases, repositories and SwiftUI screens from designs I specified, wrote and refined the test suite, reviewed the layering for violations, and drafted documentation.

Its output was checked rather than accepted. One proposed rule would have refused to remove an inspection area while it still held evidence, on the assumption that removing the area would delete the photographs. Reading the Core Data model showed the assumption was wrong: the condition item to evidence relationship uses `nullify` rather than `cascade`, so removing an area detaches the evidence instead of destroying it. The rule was rejected, and the behaviour I kept preserves the evidence and reports how much of it is now unfiled.

Verification came from running the project rather than from reading the suggestions. Builds, the unit tests, the UI tests and inspection of the Core Data model and the App Group container are what caught the problems that mattered, including a crash in the share extension's collector and a Core Data warning about duplicate entity descriptions.

References

Apple. (n.d.-a). *Core Data*. Apple Developer Documentation.

<https://developer.apple.com/documentation/coredata>

Apple. (n.d.-b). *UIKit*. Apple Developer Documentation.

<https://developer.apple.com/documentation/uikit>

NSW Government. (n.d.). *Rental property condition reports*. <https://www.nsw.gov.au/housing-and-construction/rules/rental-property-condition-reports>

Tenants' Union of NSW. (2025, August). *Starting a tenancy*. <https://www.tenants.org.au/factsheet-02-starting-a-tenancy>